

E-Cart Analytics: Modernización hacia un Data Lakehouse en Databricks

Propósito del Proyecto: Este repositorio resuelve el caso de negocio "*Modernización de la Plataforma de Datos - E-Cart Analytics*". Su objetivo principal es migrar la carga analítica de una base de datos transaccional (PostgreSQL) que estaba sufriendo problemas de rendimiento, hacia una arquitectura moderna, escalable y gobernada en la nube utilizando **AWS** y **Databricks Unity Catalog**.

1. Contexto del Negocio

El Problema Original: La empresa "E-Cart" operaba todo su comercio electrónico sobre una base de datos PostgreSQL. Con el rápido crecimiento de la empresa, los equipos de negocio (Finanzas y Marketing) comenzaron a ejecutar reportes pesados directamente sobre esta base de datos transaccional. Esto causaba caídas del sistema, enlentecía la aplicación para los usuarios finales y no permitía hacer análisis predictivos.

La Solución: Diseñar un **Data Lakehouse**. Extraer la información desde PostgreSQL sin afectar su rendimiento y llevarla a la nube (AWS S3). Una vez allí, utilizar el poder de procesamiento distribuido de **Databricks (Apache Spark)** para limpiar, transformar y organizar los datos, preparándolos para ser consumidos por tableros estadísticos (Power BI, Tableau) de manera súper rápida e independiente de la aplicación principal.

 De un concepto teórico a una plataforma funcional

este repositorio contiene un pipeline 100% funcional. Se desarrolló un simulador avanzado en Python (`bronze/generate_dummy_data.py`) utilizando la librería **Faker** para generar decenas de miles de registros sintéticos (con nombres latinos, correos y transacciones coherentes), permitiendo probar la ingesta y transformación en un entorno de Big Data realista.

```

generate_dummy_data.py 2, M X
spinbyoxxo-data-pipeline > bronze > generate_dummy_data.py > ...
6
7 # Configurar Faker para generar nombres y datos en español latino
8 fake = Faker('es_MX')
9
10 NUM_USERS = 10000
11 NUM_ORDERS = 30000
12
13 print("Generando datos para E-Card Analytics...")
14
15 # ----- Tabla USERS -----
16 users_data = []
17 countries = ['GT', 'SV', 'HN', 'CR', 'CO', 'MX', 'CL'] # Países de la región
18
19 for _ in range(NUM_USERS):
20     created = fake.date_time_between(start_date='-2y', end_date='-1m')
21     updated = created + timedelta(days=random.randint(0, 30))
22
23     users_data.append({
24         'id': str(uuid.uuid4()),
25         'name': fake.name(),
26         'email': fake.unique.email(),
27         'country': random.choice(countries),
28         'created_at': created.strftime('%Y-%m-%d %H:%M:%S'),
29         'updated_at': updated.strftime('%Y-%m-%d %H:%M:%S')
30     })
31
32 df_users = pd.DataFrame(users_data)
33
34 # ----- Tabla ORDERS -----
35 orders_data = []

```

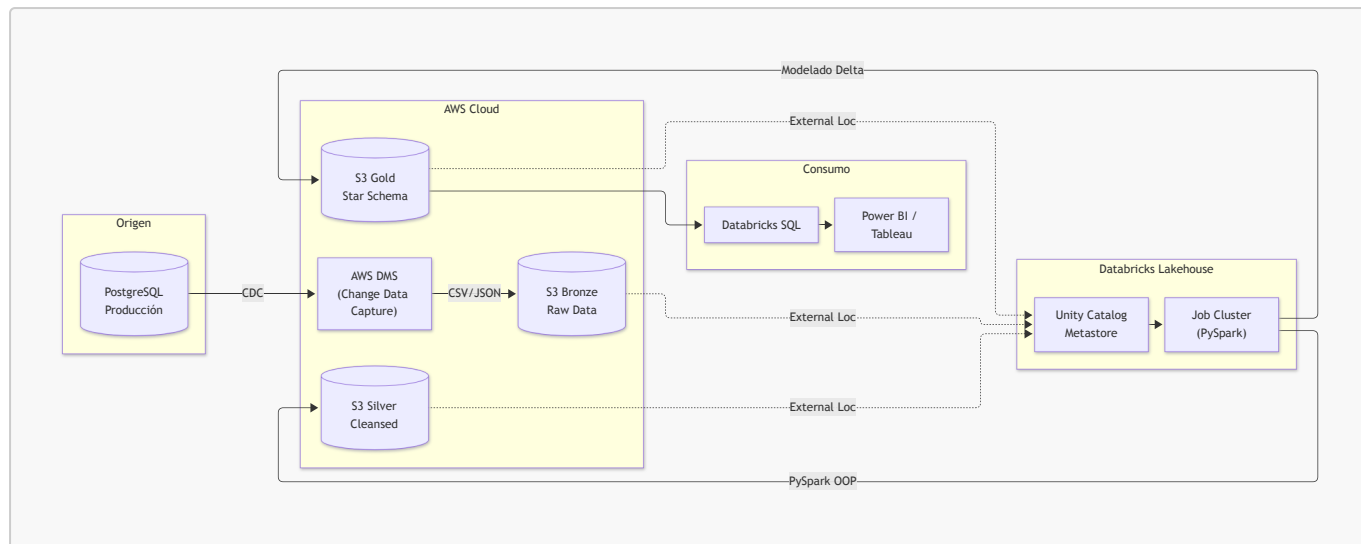
Script generador de datos transaccionales, creando perfiles de usuarios realistas para alimentar la capa Bronze y probar el rendimiento del pipeline.

2. Arquitectura de la Solución (End-to-End)

El flujo de datos implementado sigue el estándar de la industria conocido como **Arquitectura Medallion**, dividiendo los datos en tres capas de pureza:

- 🏅 **Capa Bronze (Datos Crudos):** Almacena los archivos exactos como llegaron de las fuentes transaccionales (`users`, `orders`, `order_items`). Es un respaldo inmutable histórico.
- 🥈 **Capa Silver (Datos Limpios):** Aquí la data cobra sentido comercial. Se tipifican las columnas, se eliminan registros corruptos y, **muy importante**, se encriptan (enmascaran) los datos personales de los clientes (PII) para cumplir con leyes de privacidad de datos.
- 🥇 **Capa Gold (Modelos de Negocio):** Los datos limpios se cruzan y consolidan en modelos para el negocio (*Star Schema*). Aquí nacen las tablas que usarán los analistas (ej. `dim_users` y `fct_sales`).

Diagrama General de Arquitectura Cloud



3. Gobernanza, Privacidad y Seguridad

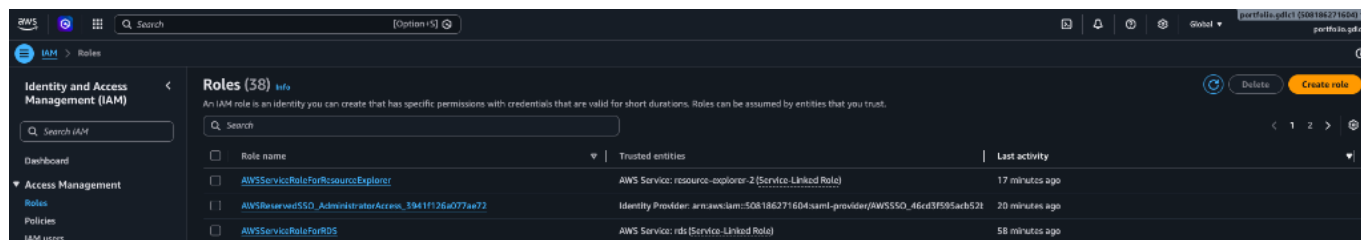
Este proyecto fue diseñado con la seguridad en primer plano, no como una idea de último momento:

1. **Protección de Datos Personales (PII):** Durante la transformación en la capa Silver (`st_users`), los correos electrónicos de los usuarios son encriptados mediante un algoritmo criptográfico de una sola vía (**SHA-256**). Esto permite a los analistas saber que es un usuario único, pero no pueden ver su correo real.
2. **Databricks Unity Catalog:** Actúa como el gran "gobernante" de los datos. Unity Catalog vincula las carpetas físicas de S3 con Databricks, permitiendo gestionar permisos a nivel de tablas, esquemas e incluso columnas.

🔑 Gestión de Accesos (AWS IAM Roles)

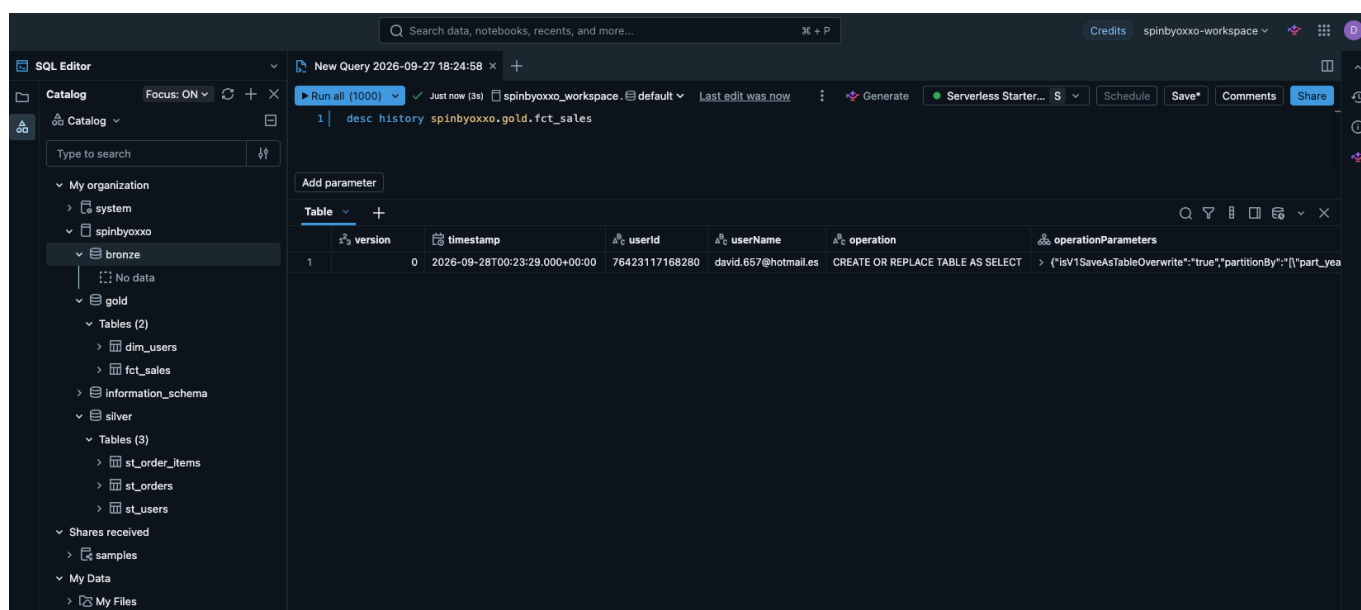
La arquitectura sigue el principio de **menor privilegio**, aislando las responsabilidades en diferentes Roles de IAM administrados por Terraform (todos prefijados con `spinbyoxxo-` para fácil identificación en tu consola de AWS):

- **Cross-Account Role** (`spinbyoxxo-databricks-cross-account-role`): Permite al plano de control global de Databricks instanciar clústeres EC2 (las máquinas virtuales que ejecutan Spark) en tu VPC de AWS de manera segura usando `External ID`.
- **UC Metastore Access Role** (`spinbyoxxo-uc-access`): Rol exclusivo para que el Metastore de Unity Catalog pueda administrar sus metadatos internos en su bucket S3 dedicado (`spinbyoxxo-databricks-metastore-...`).
- **UC Storage Credentials Role** (`spinbyoxxo-uc-storage-role`): Rol crítico que cuenta con políticas `self-assuming` estrictas. Es utilizado por Databricks para leer y escribir los datos reales directamente en los Data Lakes (los buckets Bronze, Silver y Gold).



Políticas de IAM provisionadas desde código, garantizando la seguridad entre cuentas (Cross-Account).
 Nota: En tu consola verás roles nativos de AWS (como AWSServiceRoleForRDS), pero los roles de esta arquitectura siempre inician con el prefijo del proyecto.

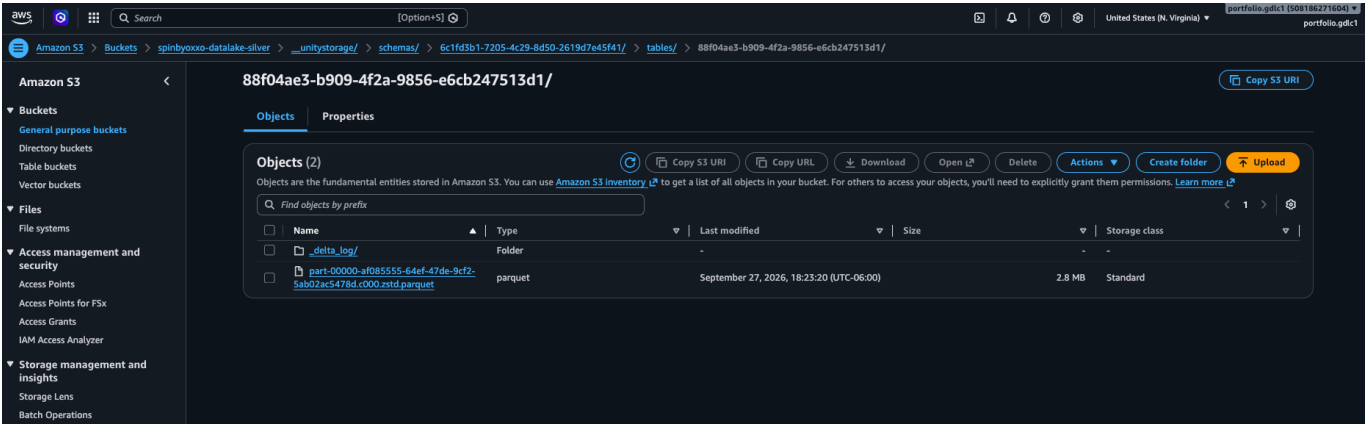
- Formato Delta Lake:** Todas las tablas de las capas Silver y Gold se guardan nativamente en formato **Delta**. Esto permite a la empresa hacer "viajes en el tiempo" (recuperar datos borrados) y asegura que, si un proceso de transformación falla a la mitad, los datos no se corrompan (transacciones ACID).



Demostración de las capacidades transaccionales de Delta Lake (**DESC HISTORY**), mostrando el registro de auditoría de quién y cómo se modificó la tabla.

Almacenamiento Físico en AWS S3

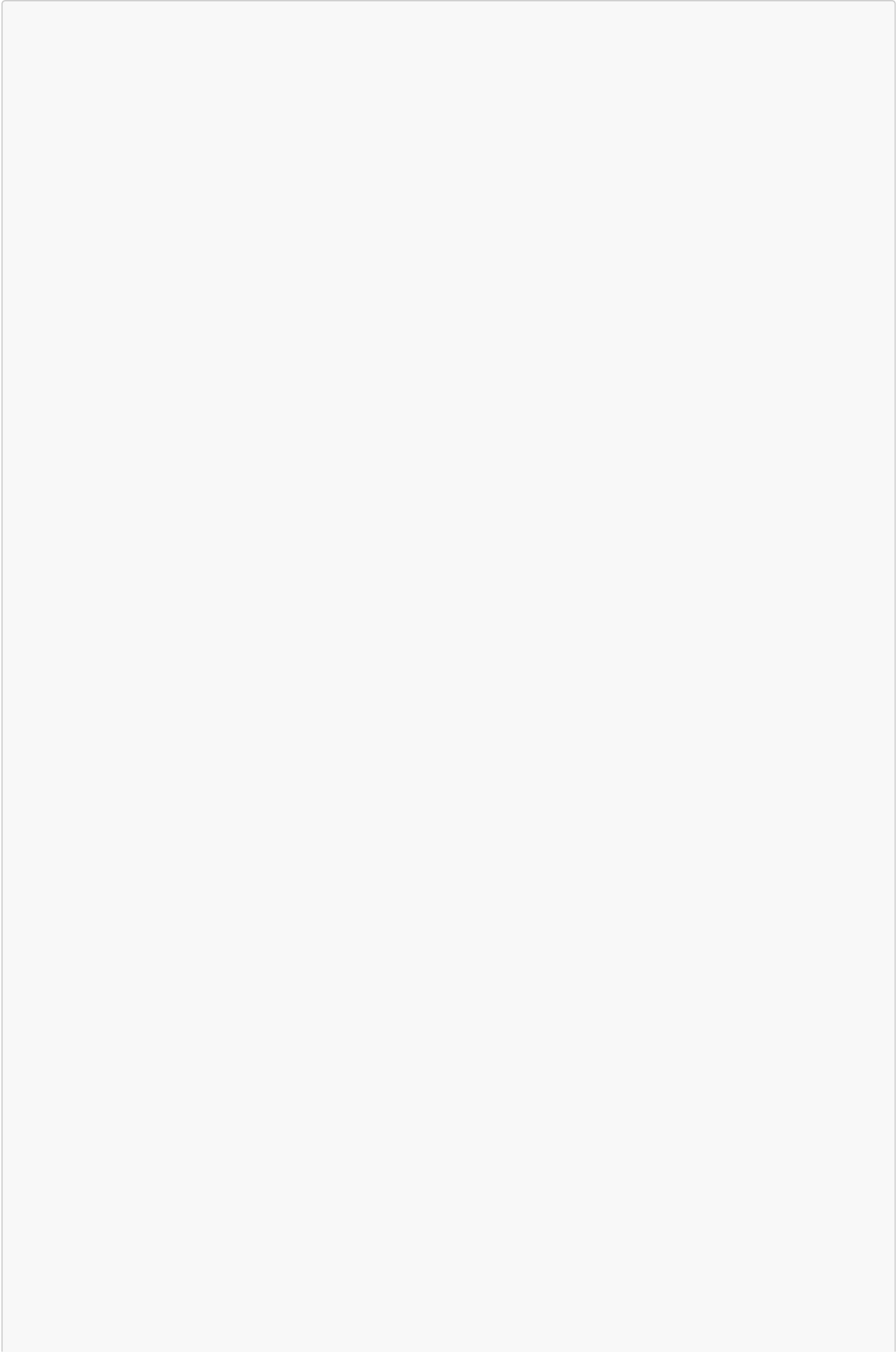
Aunque los analistas interactúan con tablas en Databricks, los datos reales residen de manera segura en tu propia cuenta de AWS. Unity Catalog abstrae la complejidad organizando los archivos transaccionales (**.parquet** y **_delta_log**) bajo identificadores únicos (UUIDs).



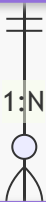
Almacenamiento nativo de las tablas Delta dentro del bucket S3 (Capa Silver), administrado y gobernado transparentemente por Unity Catalog.

Diseño de Datos (Capa Silver)

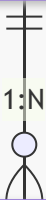
En la capa Silver los datos se limpian, enmascaran y tipifican, pero conservan la estructura transaccional original antes del modelado analítico:



st_users			
string	user_id	PK	
string	name		
string	email_hash		PII Enmascarada
string	country		
timestamp	created_at		
timestamp	updated_at		



st_orders		
string	order_id	PK
string	user_id	FK
decimal	total_amount	
string	status	
date	order_date	
timestamp	updated_at	



st_order_items		
string	item_id	PK
string	order_id	FK
string	product_id	
decimal	quantity	

	decimal	price	
--	---------	-------	--

Diseño del Modelo de Datos (Capa Gold)

dim_users		
string	user_id	PK
string	name	
string	email	
string	country	
timestamp	created_at	
timestamp	updated_at	
boolean	is_active	



fct_sales		
string	id	PK
string	order_id	
string	user_id	FK
string	product_id	
decimal	total_amount	
decimal	quantity	
decimal	price	
string	status	
date	order_date	
int	part_year	
int	part_month	

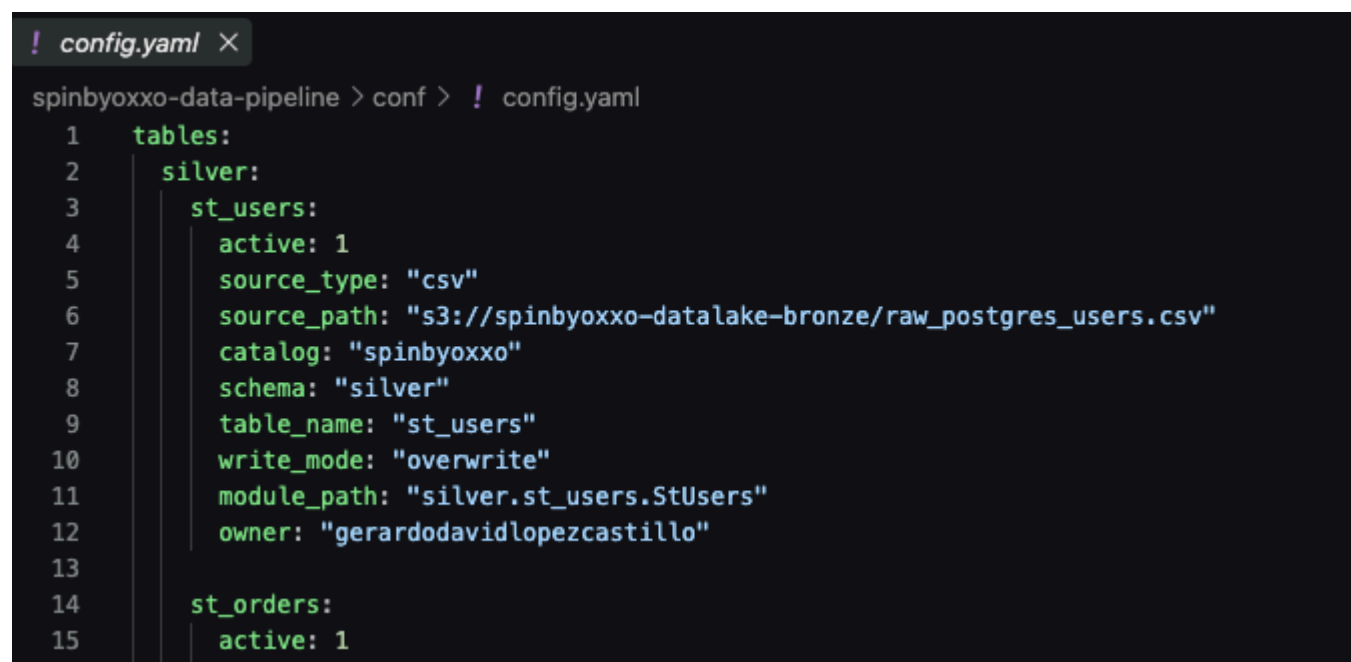
(La tabla `fct_sales` está particionada físicamente en el almacenamiento en la nube por `part_year` y `part_month` para acelerar drásticamente los reportes que buscan ventanas de tiempo específicas).

SCD Tipo 2 (Slowly Changing Dimensions): Como se aprecia en la tabla `dim_users`, se implementó el campo `is_active` (booleano). Esto prepara el modelo de datos para retener el historial completo de cambios de un usuario (por ejemplo, si cambia de país). Cuando un dato cambia, el registro antiguo se marca como inactivo (`is_active = False`) y se inserta el nuevo como activo, permitiendo que las ventas históricas de `fct_sales` apunten siempre a la fotografía correcta del usuario en el momento de la compra.

4. Ingeniería Híbrida: Código Limpio y Escalable

El código de transformación no es un script gigante difícil de leer. Fue diseñado combinando **Programación Orientada a Objetos (OOP)** y **Jupyter Notebooks**, logrando lo mejor de ambos mundos:

- **La Clase Maestra (`src/etl_master.py`):** Contiene la lógica dura y repetitiva. Se encarga de extraer, cargar en formato Delta y particionar, todo leyendo desde el `config.yaml`.
- **Los Notebooks Analíticos (`silver/*.ipynb`, `gold/*.ipynb`):** Cada tabla del negocio tiene su propio Notebook. Estos heredan de la clase maestra, por lo que su código es extremadamente limpio (solo declaran cómo se transforma el dato). Al ser Notebooks, los analistas de datos en la nube pueden abrirlos visualmente en Databricks y explorar las celdas sin tener que entender arquitecturas complejas de software.
- **Configuración Dinámica (`conf/config.yaml`):** Una sola "consola de mandos" que enciende, apaga o redirige tablas completas con cambiar una sola palabra, sin tocar el código fuente de Python.



```
! config.yaml X
spinbyoxxo-data-pipeline > conf > ! config.yaml
1  tables:
2    silver:
3      st_users:
4        active: 1
5        source_type: "csv"
6        source_path: "s3://spinbyoxxo-datalake-bronze/raw_postgres_users.csv"
7        catalog: "spinbyoxxo"
8        schema: "silver"
9        table_name: "st_users"
10       write_mode: "overwrite"
11       module_path: "silver.st_users.StUsers"
12       owner: "gerardodavidlopezcastillo"
13
14     st_orders:
15       active: 1
```

Un solo archivo YAML controla toda la orquestación: metadatos, esquemas (Silver/Gold), ubicaciones y modos de escritura.

5. Infraestructura como Código (Terraform)

Crear recursos de manera manual en Amazon Web Services (AWS) dando clics es una mala práctica propensa a errores humanos. Este proyecto cuenta con la carpeta `infrastructure/terraform/`, donde toda la infraestructura está automatizada en código. Con un solo comando (`terraform apply`), la nube crea de forma idéntica y segura:

- La red virtual (VPC, Subredes).
- Los contenedores S3 de almacenamiento.
- Los Permisos y Roles (IAM).
- El Workspace de Databricks y Unity Catalog.

```

infrastructure/terraform
├── .terraform
├── .terraform.lock.hcl
├── databricks_catalog.tf
├── databricks_iam.tf
├── databricks_network.tf
├── databricks_storage.tf
├── databricks_unity_catalog.tf
├── databricks_workspace.tf
├── network.tf
├── outputs.tf
├── provider.tf
├── s3.tf
├── terraform.tfstate
├── terraform.tfstate.backup
├── terraform.tfvars
└── variables.tf

Problems Output Debug Console Terminal Ports
(prjct_de_globalcase) + terraform git:(main) terraform apply
Apply complete! Resources: 2 added, 4 changed, 2 destroyed.

Outputs:
databricks_workspace_url = "https://dbc-8d53a6ea-4124.cloud.databricks.com"
processed_bucket_name = "spinbyoxxo-datalake-silver"
raw_bucket_name = "spinbyoxxo-datalake-bronze"
(prjct_de_globalcase) + terraform git:(main) terraform apply
data.databricks_aws_assume_role_policy.this: Reading...
data.databricks_aws_crossaccount_policy.this: Reading...
data.databricks_aws_assume_role_policy.this: Read complete after 0s [id=815efla5-3a14-4b5e-8e5a-0691d526e07a]
data.databricks_aws_crossaccount_policy.this: Read complete after 0s [id=]
data.databricks_user.me: Reading...
data.databricks_user.me: Read complete after 0s [id=72954720769790]
data.aws_iam_policy_document.passrole_for_uc: Reading...
aws_sip_nat_eip: Refreshing state... [id=eipalloc-0318cc408525db099]
aws_vpc.main: Refreshing state... [id=vpc-0b6a7d6d2305e0a07]
aws_iam_role.uc_storage_role: Refreshing state... [id=spinbyoxxo-uc-storage-role]
aws_iam_role.cross_account_role: Refreshing state... [id=spinbyoxxo-databricks-cross-account-role]
aws_s3_bucket.metastore: Refreshing state... [id=spinbyoxxo-databricks-metastore-508186271604]
aws_s3_bucket.raw_data: Refreshing state... [id=spinbyoxxo-datalake-bronze]
aws_s3_bucket.processed_data: Refreshing state... [id=spinbyoxxo-datalake-silver]
aws_s3_bucket.root_storage_bucket: Refreshing state... [id=spinbyoxxo-databricks-root-bucket-508186271604]
aws_s3_bucket.athena_results: Refreshing state... [id=spinbyoxxo-athena-results]

```

Despliegue de los recursos ejecutando `terraform apply` desde la terminal integrada, mostrando la organización modular de los archivos `.tf`.

Nota Técnica de Optimización: En iteraciones previas se probaron tecnologías antiguas (AWS Redshift, AWS ECS Fargate, ECR Docker). Al migrar a Databricks, se depuraron y destruyeron todos esos recursos obsoletos, demostrando un enfoque **FinOps** (optimización de costos) para no dejar infraestructura fantasma consumiendo presupuesto.

6. Guía de Ejecución Rápida

La mayor ventaja de esta arquitectura es su **Modelo Dual**. Puedes probar tu código localmente sin gastar dinero en la nube, y luego subir el mismo código a producción sin modificar una sola línea.

A) Ejecución Local (Para Desarrolladores / Pruebas)

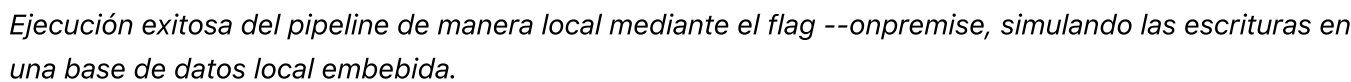
1. Instala el entorno virtual con las dependencias:

```
pip install -r requirements.txt
```

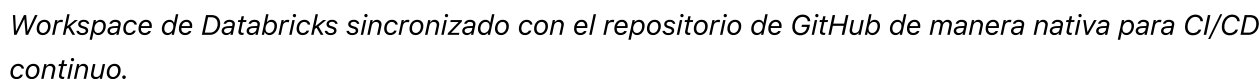
2. Ejecuta el orquestador principal con la bandera de pruebas:

```
python main.py --onpremise
```

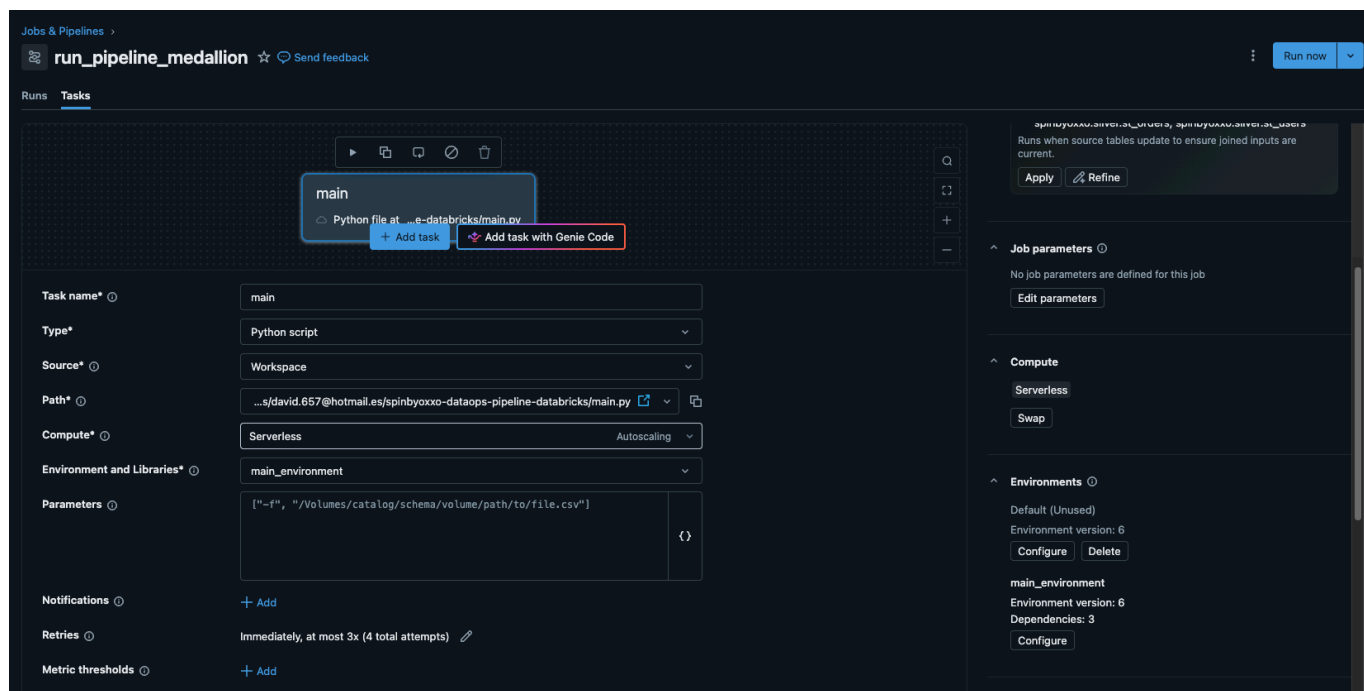
¿Qué hace la bandera `--onpremise`? Intercepta la configuración, simula la existencia de Unity Catalog dentro de tu computadora creando la base de datos `spark-warehouse`, y lee los CSV locales en la carpeta `bronze/` en lugar de ir a buscarlos al S3 de AWS.



1. En tu espacio de trabajo (Workspace) de Databricks, usa la función **Databricks Repos (Git Folders)** para conectar este repositorio de GitHub.
2. Databricks clonará el proyecto idéntico a la nube.

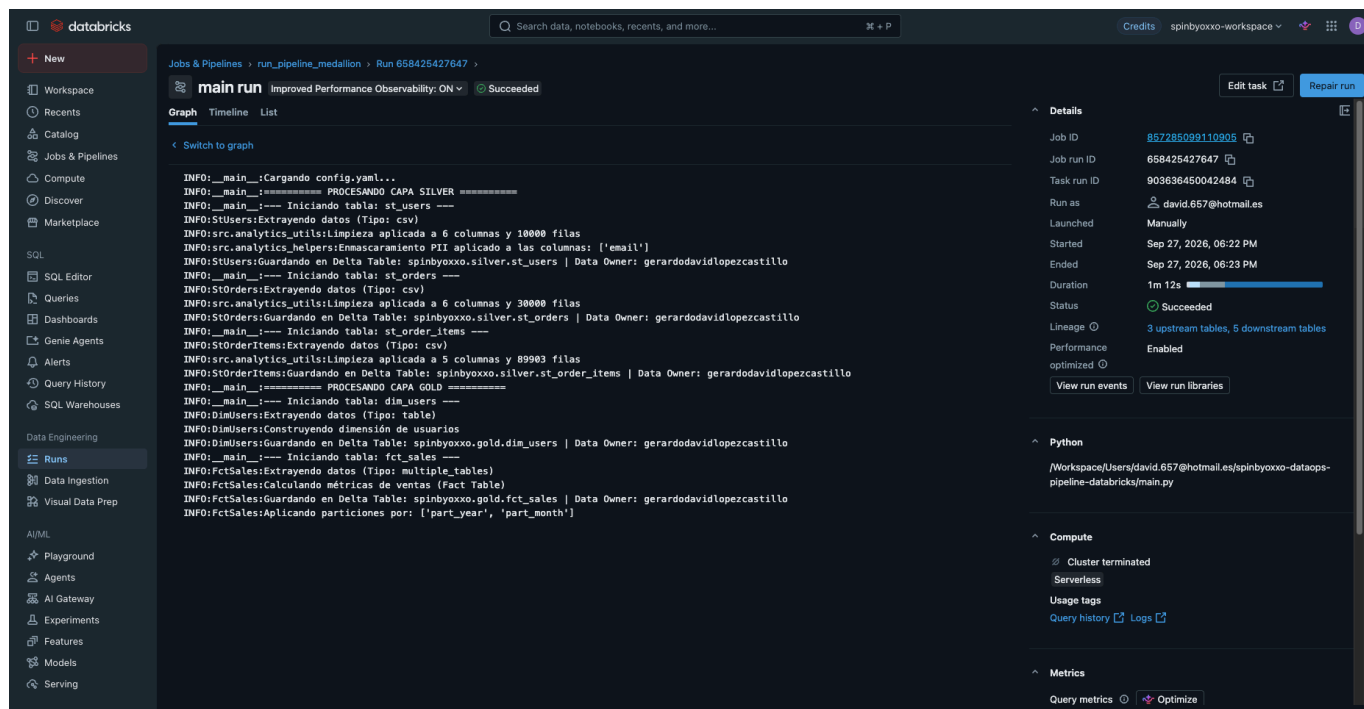


3. Dirígete a la pestaña **Workflows -> Create Job**.
4. Crea una tarea de tipo **Python Script** apuntando al archivo **main.py** de tu repositorio recién sincronizado.
5. En la configuración del Clúster del Job, asegúrate de instalar las librerías **omegacnf** y **ipynb**.



Configuración de la tarea en Databricks Workflows apuntando al script principal y definiendo los parámetros de cómputo serverless.

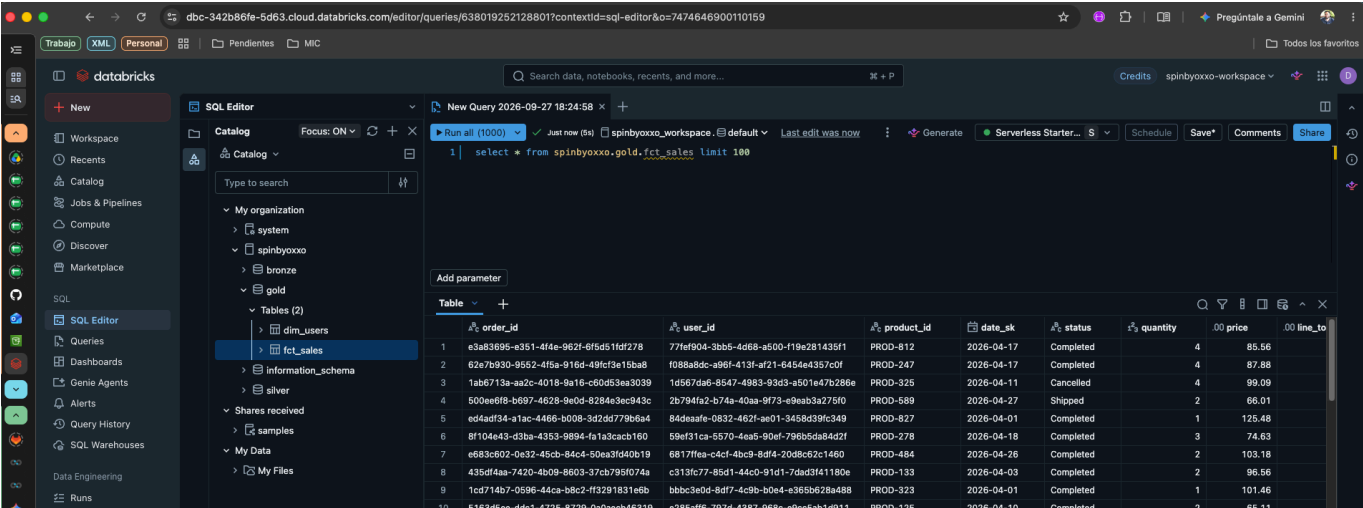
6. Presiona **Run Now**. Al no enviar la bandera **--onpremise**, el script utilizará los recursos empresariales: leerá los terabytes de datos en AWS S3 y guardará las tablas en Unity Catalog.



Ejecución 100% exitosa del pipeline completo orquestado mediante un Job Cluster automatizado.

 Consumo de Datos (Databricks SQL)

Una vez completado el pipeline, la arquitectura permite de manera instantánea realizar analítica de datos utilizando Databricks SQL Editor o conectándolo con herramientas de BI (PowerBI, Tableau, etc.).



Consulta de las tablas de negocio finales de la capa Gold utilizando Databricks SQL Engine.

Gerardo López

Senior Data Engineer | Data Architect | Cloud Engineer

 LINKTREE

 LINKEDIN

 GMAIL

 GITHUB